

Projet Intégrateur : PacMan en réseau

Benoît LITAMPHA | Auriane PETER | Thomas COUTANT

Université Marie & Louis Pasteur, UFR Sciences et Techniques

Licence CMI Informatique – 3^{ème} année
2025–2026

Encadrant : Julien Bernard



Plan

Présentation et organisation

Serveur

Client

Réseau

Bilan et conclusion

Démonstration

Plan

Présentation et organisation

Serveur

Client

Réseau

Bilan et conclusion

Démonstration

Introduction

Contexte

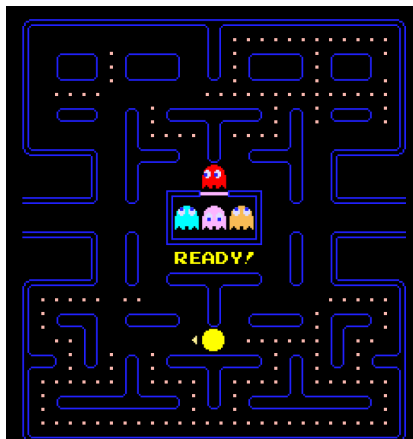
- Années 80
- Jeu d'arcade

Gameplay

- Labyrinthe
- Pac-gommes
- Fantômes
- Mode chasse (power-up)

Projet

- Jeu réseau
- Génération procédurale
- IA des fantômes



Pac-Man (original)

Adaptation du projet :

- 1 joueur Pac-Man
- Plusieurs joueurs Fantômes
- Fantômes contrôlés par **IA** si joueurs insuffisants
- Cabane comme lieu d'apparition des fantômes

Déroulement d'une partie :

Connexion → Liste des Salons → Salon → Jeu → Fin de jeu

- **C++** avec le framework **GameDev Framework**
- **Git** via GitHub
- **Réunions régulières** : toutes les 1 à 2 semaines
- Communication quotidienne avec **Discord**
- Évolution **incrémentale**



■ Auriane PETER

- Développement du client
- Interface et interactions du joueur

■ Benoît LITAMPHA

- Communication client–serveur
- Support sur client et serveur

■ Thomas COUTANT

- Développement du serveur
- Conception de la logique du jeu

Plan

Présentation et organisation

Serveur

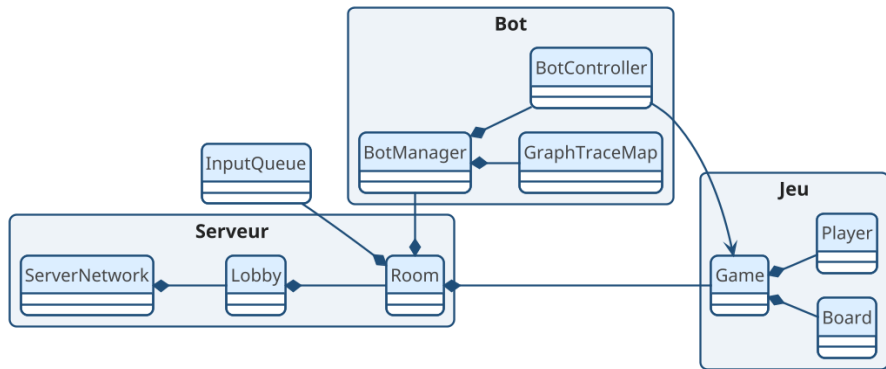
Client

Réseau

Bilan et conclusion

Démonstration

Conception



Client → Serveur → Hall → Salon → Game

Architecture générale

Architecture

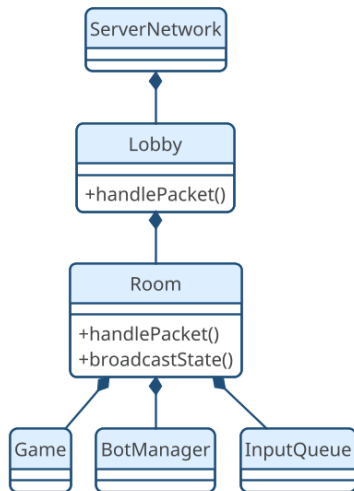
- Organisation **modulaire**
- Séparation des responsabilités
- Lisibilité et maintenabilité

Couches

- Réseau → Serveur
- Organisation → Hall
- Partie → Salon
- Logique → Jeu

Rôle central du salon

- Coordination des composants
- Gestion des joueurs
- Synchronisation des états



Boucle de jeu

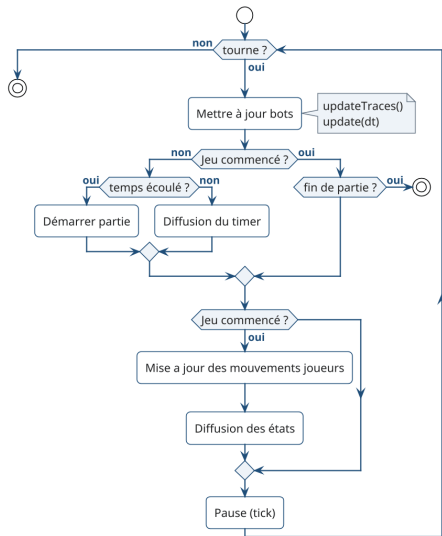
- Serveur **autoritaire**
- Affichage **côté client**
- Anti-triche et synchronisation

Tick serveur

- Mise à jour **périodique**
- Thread serveur :
 - Positions
 - Collisions
 - Scores et timers

États

- Pré-jeu
- Partie en cours
- Fin de partie



IA des fantômes - Principe

Objectif

- Adaptation au nombre de joueurs
- Intelligence collective

Inspiration : fourmis

- Phéromones
- Communication indirecte
- Suivi de traces

Implémentation

- Traces sur le plateau
- Paramètres :
 - Intensité (décroissance)
 - Propriétaire
 - Couleur

```
struct Trace {  
    TraceType type;  
    uint32_t ownerId;  
    float intensity;  
};
```

Rouge : Pac-Man

| Bleu : fantômes

IA des fantômes - Prise de décision

Calcul du score

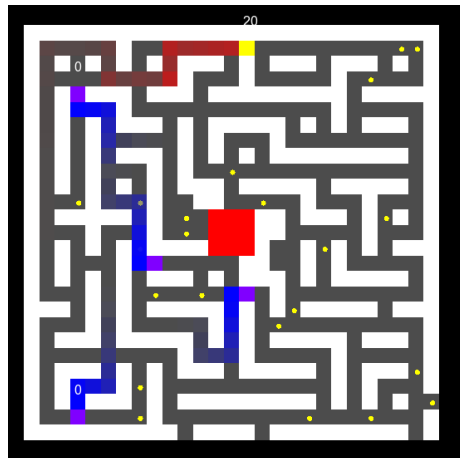
$$\text{Score} = \text{Attraction} - \text{Répulsions}$$

Influences

- Pac-Man → Attraction
- Fantômes → Répulsion
- Répulsion ++ pour lui-même
- Vision → priorité Pac-Man
- Tracking → BFS

Intégration serveur

- Bots = joueurs
- Envoi d'inputs
- InputQueue → Salon



Génération procédurale du plateau

Algorithme

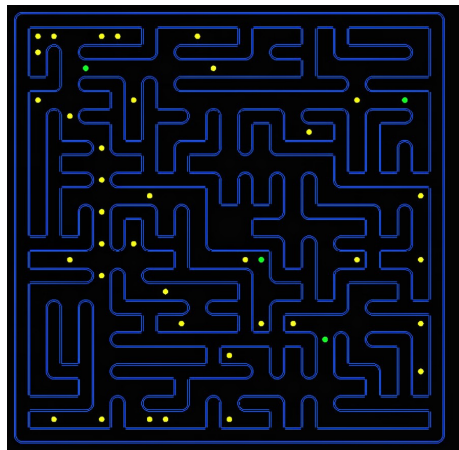
- Variante de Prim

Étapes

- Plateau = murs
- Case de départ
- Liste des murs adjacents
- Ouverture aléatoire

Propriétés

- Connexe
- Aucune zone isolée

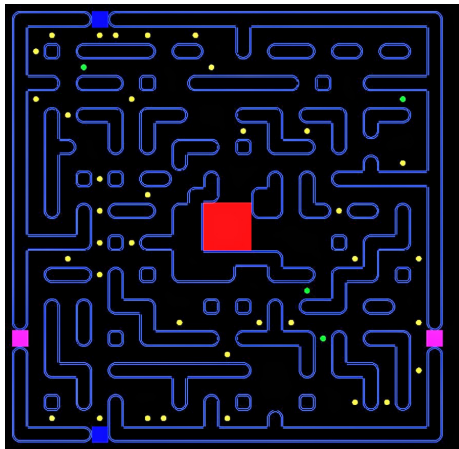


Contraintes

- Zone centrale : hut 3×3
- Accès $\rightarrow$ coins de départ

Améliorations

- Cycles
- Moins de culs-de-sac
- Portails latéraux



Plan

Présentation et organisation

Serveur

Client

Réseau

Bilan et conclusion

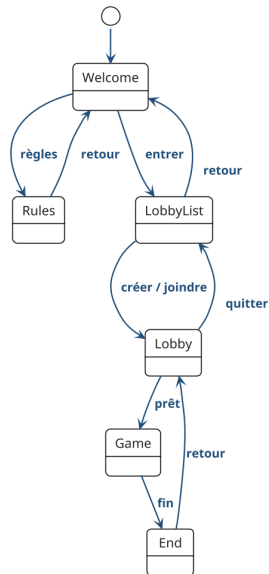
Démonstration

Fonctionnement du gestionnaire de scènes

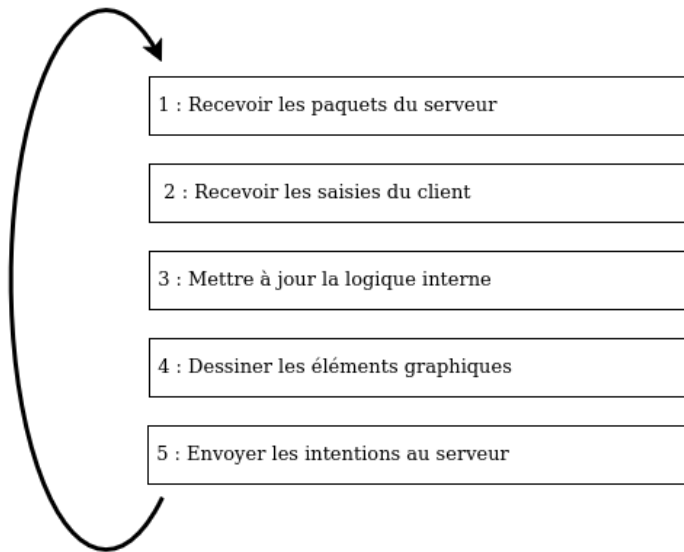
Gère quelle scène s'affiche via une pile

- Une étape de jeu = **une scène**
- **Autonomie** des scènes
- **Séparation** entre logique / rendu

→ Met aussi en place le réseau



Boucle de jeu client



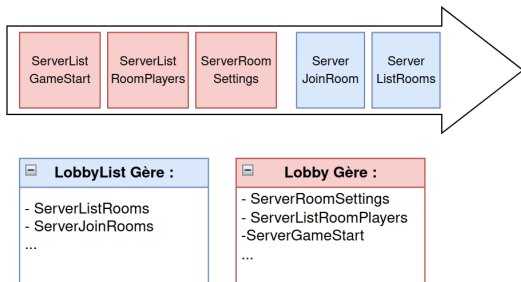
Boucle de jeu

Réception dans un **Thread dédié**

Stockage des paquets dans une file partagée :

- **Consommation** des paquets selon leur type
- **Gestion** des éléments concernés uniquement

Mise à jour des scènes selon les paquets reçus



Système d'Événements :

- Mapping clavier → actions (up, down, left, right)
- Passage de la souris
- Clics de la souris
- Redimension de la fenêtre

Envoi partiel des entrées au serveur

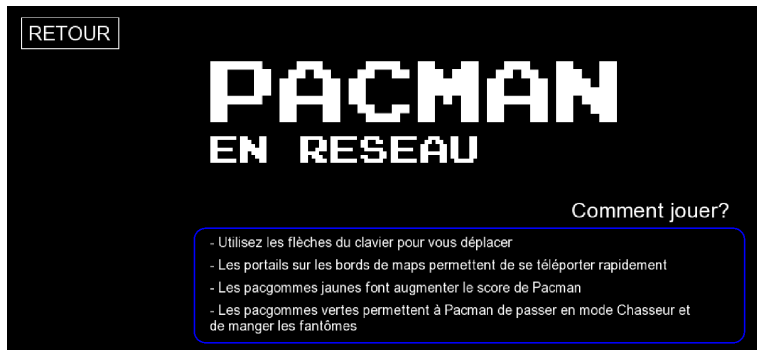
Room n° 1

Paramètre du jeu

Joueurs max :	-	2	+
Nb de bots :	-	4	+
Temps de jeu (secondes) :	-	120	+
Nb. PV Pacman :	-	1	+
Nb. PV Fantômes :	-	1	+

Liste des joueurs (1 / 2) :

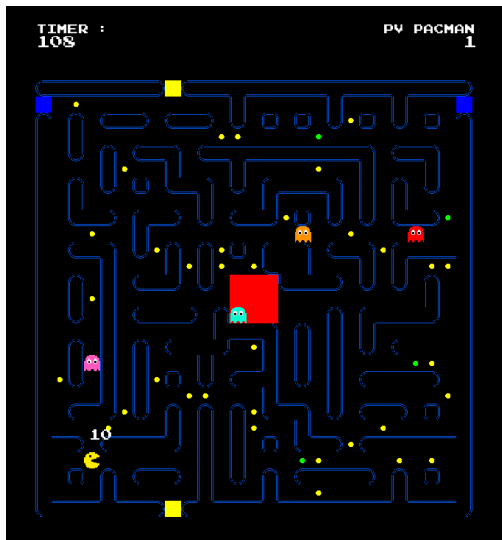
 player_1 (vous) (pas prêt)



Une entité par scène

Entités à **responsabilité limitée** :

- Traduit la logique en **éléments graphiques concrets**
- Interagit directement avec le client : **clics client** → **scène**



Envoi de messages des scènes
au serveur

Deux modèles :

- événementiel (clics de souris)
- périodique (mouvements en jeu)

Principe d'envoi **commun**

Transmission de l'intention du
client au serveur

Plan

Présentation et organisation

Serveur

Client

Réseau

Bilan et conclusion

Démonstration

Communication avec socket

- Choix entre UDP et TCP : TCP choisit pour sa fiabilité

Classes spécialisées

- **ServerNetwork** et **ClientNetwork** pour la réception des paquets

Multi-Threading

- Code de **ServerNetwork** et **ClientNetwork** exécutés en parallèle de la boucle principale

Messages : structures C++

- Connues par le client et le serveur
- Contiennent des structures *communes*
- Ont une constante *type* pour les identifier lors de la désérialisation
- Ont une nomenclature spécifique :
 - *ServerXXX* = envoyée par le serveur
 - *ClientXXX* = envoyée par le client

Sérialisation/désérialisation

- Nécessaire pour l'envoi/réception
- **Opérateur** | avec un type *Archive*

```
struct PlayerData
{
    uint32_t id;
    float x, y;
    std::string name;
    PlayerRole role;
    int score;
    bool ready = false;
    unsigned int hp = 0;
};
```

```
struct ServerListRoomPlayers
{
    static constexpr gf::Id type = "
        ServerListRoomPlayers"_id;
    std::vector<PlayerData> players;
};
```

```
template <typename Archive>
Archive &operator|(Archive &ar,
    ServerListRoomPlayers &data)
{
    return ar | data.players;
}
```

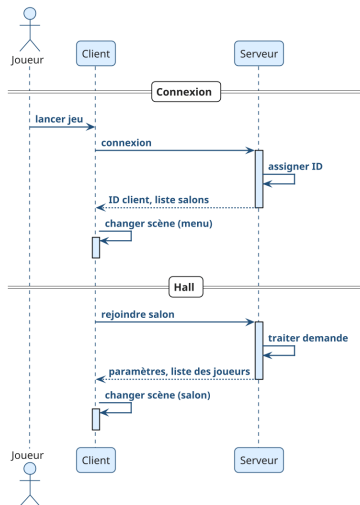
Structure des échanges

Connexion

- **Client** : Demande de connexion
- **Serveur** :
 - Assignment d'un id
 - Envoi l'id + liste des salons

Hall

- **Client** : Demande de création/connexion à un salon
- **Serveur** :
 - Traitement + envoi de la confirmation
 - **Diffusion** liste des joueurs/paramètres



Structure des échanges

Salon

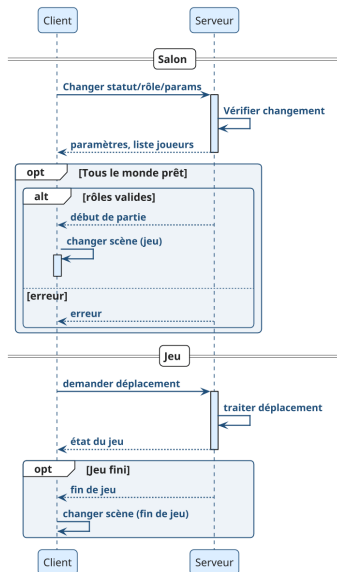
- **Client** : Demande de changement de rôle/statut/paramètres
- **Serveur** :
 - Vérifie la validité
 - **Diffusion** liste des joueurs ou paramètres

Tous les joueurs prêts ? : **diffusion** début de partie/erreur

En jeu

- **Client** : Demande de déplacement
- **Serveur** :
 - Vérifie, gère les conséquences
 - **Diffusion** état du jeu

Si fin de jeu : **diffusion** fin du jeu



Plan

Présentation et organisation

Serveur

Client

Réseau

Bilan et conclusion

Démonstration

Objectif principal atteint

- Jeu en réseau
- Intelligence artificielle des fantômes
- Génération procédural de la carte

Acquisition et renforcement des compétences

- Programmation réseau, multi-threadé
- Modélisation des systèmes
- Travail en équipe

Perspectives

- Plusieurs rôles fantômes
- Intelligence artificielle pour Pac-Man
- Salon privé

Plan

Présentation et organisation

Serveur

Client

Réseau

Bilan et conclusion

Démonstration

**Merci de votre attention.
Des questions ?**